04

Gestão de Base de Dados e Concorrência

Gestão de Base de Dados e Concorrência

Como proteger dados quando várias operações acontecem ao mesmo tempo.

Roteiro da aula

- Problema real no projeto base
- Concorrência em linguagem simples
- Transações de banco
- Atomic Locks
- Lock no banco
- Constraints também protegem
- Estratégias de indexação
- Identificação de bottlenecks
- Pruning Models

Problema real no projeto base

Cenário
Duas recepcionistas tentam agendar pacientes diferentes para o mesmo profissional no mesmo horário.

Risco
O sistema pode criar dois agendamentos para o mesmo horário se a verificação e a criação não forem protegidas.

Concorrência em linguagem simples

Concorrência acontece quando duas ou mais operações disputam o mesmo dado.

Exemplo

- Usuário A verifica horário livre.
- Usuário B verifica o mesmo horário livre.
- Usuário A grava o agendamento.
- Usuário B também grava.

Resultado

Dois registros válidos isoladamente, mas inválidos juntos.

Transações de banco

Definição
Transação garante que um conjunto de operações seja concluído por inteiro ou desfeito por inteiro.

Exemplo

Uso

- criar agendamento e histórico;
- registrar pagamento e baixa;
- atualizar estoque e criar pedido.

Exemplo de código

```
DB::transaction(function () use ($data) {
    $appointment = Appointment::create([...]);
    AppointmentLog::create([...]);

    return $appointment;
});
```

Atomic Locks

Definição
Atomic lock cria uma trava temporária para impedir que duas execuções entrem no mesmo trecho crítico ao mesmo tempo.

Exemplo

Drivers

- Redis;
- database;
- memcached;
- dynamodb.

Exemplo de código

```
$lock = Cache::lock("professional:{$id}:{$startsAt}", 10);

$lock->block(5, function () {
    // verificar disponibilidade e criar agendamento
});
```

Lock no banco

Exemplo

Cuidado

Locks resolvem problemas reais, mas podem gerar espera e deadlocks se usados sem critério.

Exemplo de código

```
DB::transaction(function () use ($professionalId, $startsAt)
{
    $exists = Appointment::query()
        ->where('professional_id', $professionalId)
        ->where('starts_at', $startsAt)
        ->lockForUpdate()
        ->exists();

    if ($exists) {
        throw new TimeSlotUnavailable();
    }
});
```

Constraints também protegem

Exemplo
Criar índice único para impedir duplicidade.

Mensagem principal
A aplicação deve validar antes, mas o banco deve proteger a regra essencial.

Exemplo de código

```
$table->unique(['professional_id', 'starts_at']);
```

Estratégias de indexação

Índice é um atalho para o banco encontrar dados.

No sistema guia

- appointments.professional_id
- appointments.clinic_id
- appointments.starts_at
- combinação professional_id + starts_at
- combinação clinic_id + starts_at

Pergunta guia

"Como esta tela filtra os dados?"

Identificação de bottlenecks

Sinais de problema

- tela de agenda demora para carregar;
- query usa muitas linhas;
- filtro por data fica lento;
- dashboard faz agregações caras;
- endpoint da API piora com volume.

Ferramentas

- EXPLAIN;
- Telescope;
- logs de queries lentas;
- Pulse.

Pruning Models

Definição
Pruning remove automaticamente registros antigos ou obsoletos.

Exemplo

Uso

- logs temporários;
- tokens expirados;
- entregas antigas de webhook;
- notificações antigas.

Exemplo de código

```
final class WebhookDelivery extends Model
{
    use Prunable;

    public function prunable(): Builder
    {
        return static::where('created_at', '<=',
now()->subDays(30));
    }
}
```

Read/Write Splitting

Ideia
Usar uma conexão para escrita e réplicas para leitura.

Quando faz sentido
- alto volume de leitura;
- relatórios pesados;
- aplicação com réplicas configuradas.

Cuidado
Réplicas podem ter atraso. Depois de escrever, talvez a leitura imediata precise ir ao banco principal.

Projeto do módulo

Missão
Proteger o agendamento contra horário duplicado e preparar a base para crescer.

Entregas
- usar transação no fluxo de criação;
- aplicar atomic lock;
- criar constraint única;
- propor índices para agenda;
- criar pruning para logs antigos;
- discutir read/write splitting.

Checklist final

O aluno deve conseguir

- explicar race condition;
- usar transação;
- aplicar lock com Redis ou banco;
- criar índice único para regra essencial;
- escolher índices com base nas consultas;
- configurar pruning;
- explicar o risco de atraso em réplicas.